Kapitola 9

Základy práce so systémom MATLAB

Systém MATLAB (Matrix Laboratory), vyvíjaný firmou The Mathworks, Inc. v USA od roku 1984 je softvér, ktorý sa používa na vedeckotechnické výpočty, modelovanie, simuláciu, návrhy algoritmov, paralelné výpočty, meranie a spracovanie signálov. Zahŕňa výpočty, programovanie a vizualizáciu do užívateľsky ovládaného prostredia. Problémy a riešenia sú najčastejšie vyjadrené pomocou známych matematických vzťahov.

Základom MATLABu je **výpočtové jadro**, ktoré realizuje výpočtové operácie s maticami reálnych aj komplexných čísel. Všetky objekty v MATLABe sú považované za matice, avšak ich prvkami nemusia byť vždy len čísla alebo premenné, ale môžu to byť aj zložitejšie štruktúry. V tomto prostredí je tiež možné efektívne pracovať s vektormi.

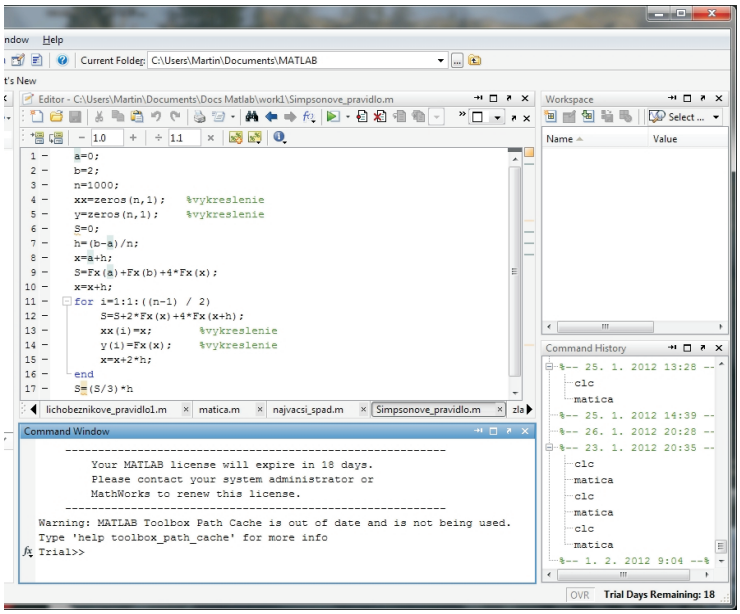
O grafické zobrazenie výsledkov výpočtov sa stará **grafický systém** MATLABu, ktorý umožňuje jednoduché vytváranie 2D aj 3D grafov s možnosťou viacerých nastavení.

MATLAB disponuje množstvom **pracovných nástrojov**, ktorými sa dajú vytvárať aplikácie s vizuálnymi komponentami ako je napríklad tlačítko, zaškrtávacie políčko a pod. Obsahuje tiež vlastný programovací jazyk a všetko potrebné k písaniu a ladeniu zdrojového kódu.

Okrem výpočtového jadra existujú pre MATLAB rôzne knižnice nazývané **toolboxy**. Toolboxy sú špecifikované svojím zameraním a po nainštaľovaní do MATLABu rozšíria jeho možnosti o svoje funkcie. Existuje viacero toolboxov s rôznym zameraním, napríklad toolbox pre spracovanie signálov, prácu s neurónovými sieťami, návrh filtrov, ekonomiku a pod. Známym a veľmi využívaným systémom je Simulink, ktorý umožňuje pracovať so všetkými funkciami ako s grafickými blokmi a tie napájať na zdroje dát a prepojovať navzájom.

9.1 Pracovné prostredie

V tejto časti sa budeme zaoberať opisom pracovného prostredia MATLABu a jeho základných nastavení. Tak, ako väčšina aplikácií, aj verzie systému MATLAB majú rozlišné pracovné prostredia, preto poznamenávame, že budeme pracovať s verziou 7.12. Po spustení systému MATLAB, či už pomocou odkazu na pracovnej ploche alebo odkazom z hlavnej ponuky programov sa zobrazí pracovné prostredie, ktoré je zobrazené na obrázku 9.1. Pracovné prostredie disponuje viacerými oknami.



Obr. 9.1: Pracovné prostredie MATLABu verzia 7.12

V strede dolu sa nachádza okno *Command Window*, v ľavej časti sú pod sebou okná *Current Folder* a *Details*. Okno *Command Window* slúži na zápis príkazov a tiež na výpis riešení a zobrazovanie systémových hlásení. V pravej časti prostredia sú pod sebou okná *Workspace* a *Command History*. Najväčším oknom v strede pracovného prostredia je *Editor*, do ktorého sa zapisuje kód pre m-súbory.
Samotný názov okna *Command History* napovedá o svojej funkcii, automaticky sa doň zapisujú všetky príkazy zadané do *Command Window* a v prípade potreby je možné z neho príkaz opätovne použiť bez nutnosti jeho zápisu do *Command Window*. Listovanie v zozname použitých príkazov je možné tiež pomocou kurzových šípiek hore a dole priamo v *Command Window*.
Okno *Workspace* sa postupne napĺňa s používaním premenných v hlavnom okne. Po dvojkliku na niektorú premennú sa zobrazia detailné informácie o premennej (obsah, rozmer, štruktúra a pod.). V okne *Current Folder* sa zobrazuje zoznam

súborov aktuálneho adresára.
Tesne pod hlavným menu je položka *Current Folder*, ktorá je implicitne nastavená na zložku *.../MATLAB7/Work* a slúži na ukladanie používateľských súborov. Je možné ju zmeniť podľa vlastnej potreby.

Nad opísanými oknami sa nachádza panel príkazov s ikonami a hlavné menu MATLABu, ktoré sú väčšine používateľov známe z iných aplikácií. S ich pomocou vieme upraviť pracovnú plochu, vytvoriť vlastnú ikonu s príkazmi, spustiť pomocníka a pod.

Zapisovanie príkazov iba do *Command Window* je nepraktické a preto je v MATLABe vstavaný editor, ktorým dokážeme vytvoriť tzv. **m-súbory**. Sú to textové súbory s príponou **.m*, obsahujúce zápis postupnosti príkazov a MATLAB ich vie v danom poradí vykonať. Spustiť **m-súbory** môžeme buď cez ponuku hlavného menu *Debug/Run* alebo pomocou klávesy *F5* alebo ikonou so zelenou šípkou umiestnenou v okne editora.

9.2 Elementárne operácie s číslami

Do *Command Window* môžeme priamo zapisovať príkazy s elementárnymi matematickými operáciami, takže sa okno javí ako kalkulačka. Pri zápise reálnych čísel sa namiesto desatinnej čiarky píše bodka. Napríklad pri zápise **8.3-40*0.17** a potvrdení klávesou **ENTER** sa vypíše *ans=1.5000*. Výsledný súčet sa vypíše vo formáte na štyri desatinné miesta, čo je štandardný formát, pokiaľ používateľ nezadá iný a *ans* je názov implicitnej premennej, do ktorej sa zapisuje výsledok operácie. Na zobrazenie viacerých desatinných miest použijeme príkaz `format long`. Príkaz stačí použiť raz a všetky čísla budú vypisované s presnosťou na štrnásť desatinných miest. Pre spätné zobrazenie na štyri desatinné miesta použijeme príkaz `format short`.
V nasledujúcich tabuľkách uvádzame prehľad základných elementárnych operátorov a funkcií MATLABu.

Prehľad elementárnych matematických operácií

Znak operácie	Zápis	Poznámka
+	3+5	súčet
−	7-4	rozdiel
*	2*3	súčin
/	33/2	podiel
^	3 ^ 5	mocnina
()	6/(3+5)	použitie zátvoriek

Prehľad základných elementárnych funkcií

Funkcia	Zápis	Poznámka
sin	sin(pi)	funkcia sínus
sinh	sinh(0.58)	funkcia hyperbolický sínus
asin	asin(1)	funkcia arcussínus
asinh	asinh(pi/3)	funkcia hyperbolický arcussínus
cos	cos(2*pi)	funkcia cosínus
cosh	cosh(0.13)	funkcia hyperbolický cosínus
acos	acos(0.4)	funkcia arcuscosínus
acosh	acosh(1.0)	funkcia hyperbolický arcuscosínus
tan	tan(pi/2)	funkcia tangens

Okrem klasického zápisu reálnych čísel môže používateľ využiť zvláštne typy čísel či konštánt. Príkladom je premenná *ans*, ktorú sme už spomínali vyššie. Zapisuje sa do nej výsledok matematickej operácie, pokiaľ používateľ neurčí zápis do inej premennej. Častou používanou konštantou je *pi*, ktorá predstavuje hodnotu Ludolfovho čísla $\pi = 3.1415926535897...$ Pri matematických výpočtoch sa môžeme stretnúť so symbolmi *inf* a *NaN*. Symbol *inf* predstavuje kladnú hodnotu nekonečna. Môže sa zobraziť napríklad ako výsledok pri delení nulou. Symbol *NaN* (z anglického Not a Number) reprezentuje hodnotu, ktorá nie je číslo. Môže sa vypísať napríklad ako výsledok operácie pri delení nuly nulou.

9.3 Premenné

Pri písaní programov sa nezaobídeme bez použitia premenných, ktorých názov pozostáva z písmen a číslic podobne ako v iných programovacích jazykoch. Názov premennej musí začínať písmenom a nesmie obsahovať medzery a písmená s diakritikou. MATLAB rozlišuje v názvoch premenných malé a veľké písmená, takže nasledujúce zápisy predstavujú dve rôzne premenné.

$$\begin{aligned} m &= 0.125 \\ M &= -14; \end{aligned}$$

Pri zápise hodnôt do premenných môžeme/nemusíme písať za priraďovacím príkazom bodkočiarku tak, ako je to zobrazené vyššie. Ak bodkočiarku nenapíšeme, obsah premennej sa do *Command Window* bude vypisovať, v prípade, že bodkočiarku zapíšeme, obsah premennej sa nevypíše. Treba však poznamenať, že v oboch prípadoch sa hodnota do danej premennej zapíše.
Ak už premennú nechceme používať, môžeme ju vymazať z pamäte príkazom `clear premenna`. Ak na miesto premennej použijeme `all`, vymaže sa obsah všetkých použitých premenných.
Okrem základného priraďovacieho príkazu môžeme medzi premennými použiť aj relačné operátory.

Relácia	Zápis	Poznámka
$<$	$m < n$	relácia menší
$>$	$n > m$	relácia väčší
$<=$	$m <= n$	relácia menší a nanajvýš rovný
$>=$	$n >= m$	relácia väčší a nanajvýš rovný
$\sim=$	$m \sim= 56$	relácia nerovná sa
$==$	$m == n$	relácia porovnania

9.4 Práca s maticami

Ako sme už v úvode naznačili, základným objektom v MATLABe je matica. Vstupné údaje zapisujeme do matice alebo vektora a preto si v tejto kapitole ukážeme ako s maticami a vektormi pracovať.
Matice v MATLABe zapisujeme do hranatých zátvoriek. Stĺpce matice oddeľujeme čiarkami alebo medzerami, riadky matice oddeľujeme bodkočiarkami. Napríklad zápisy

$$A = [1\ 2\ 3; 5\ 7\ 9] \quad \text{a} \quad A = [1,\ 2,\ 3;\ 5,\ 7,\ 9]$$

sú ekvivalentné a predstavujú maticu

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 5 & 7 & 9 \end{pmatrix}$$

V prípade, že do matice zapíšeme hodnoty iba do jedného riadku oddelené čiarkami alebo medzerami, nazývame ju riadkovým vektorom.
S maticami môžeme realizovať všetky matematické operácie ako je sčítanie, odčítanie, násobenie matíc, transpozícia.

Pri zápise matíc môžeme využiť aj špecifické vlastnosti MATLABu, ako je napríklad zápis prvkov matice pomocou dvojbodky „:“. Takýto zápis využívame

v prípade, že potrebujeme zapísať do matice príliš veľký počet riadkov alebo stĺpcov. Ako príklad ukážeme generovanie vektora, ktorý bude obsahovať čísla od -2 po 5 s krokom 0.1 a nazveme ho **os**. Zápis bude vyzerať takto:

$$os = [-2 : 0.1 : 3];$$

Prvé číslo -2 predstavuje prvý prvok a posledné číslo 3 posledný prvok vektora. Číslo 0.1 medzi dvojbodkami udáva krok. Výsledný vektor potom bude mať nasledovné súradnice:

$$os = [-2.0 \ -1.9 \ -1.8 \ -1.7 \ \ldots \ 2.5 \ 2.6 \ 2.7 \ 2.8 \ 2.9 \ 3.0]$$

Samozrejmosťou je, že pri generovaní vektorov resp. matíc môžeme namiesto čísel použiť aj matematické vzorce, čo je ukázané na nasledujúcom príklade.

$$os = [-2 * pi : pi/50 : pi];$$

Ak nepotrebujeme vypisovať obsah premenných, píšeme na konci výrazov bodkočiarku. V opačnom prípade by sa obsah vektora či matice vypisoval, avšak aj takýto zdĺhavý výpis môže používateľ prerušiť stlačením kombinácie kláves **CTRL**+**C**.

9.4.1 Základné operácie s maticami

Pri práci s operáciami na maticiach, musíme definovať aspoň dve matice. Samozrejme MATLAB obsahuje aj maticové funkcie, ktoré sú definované nad jednou maticou. Jednotlivé operácie s maticami ukážeme na príkladoch uvedených ďalej. Definujme matice $A = [1\ 2; 3\ 4]$; a $B = [5\ 6; 7\ 8]$;

- `C=A+B` súčet matíc, výsledok je matica

$$C = \begin{pmatrix} 6 & 8 \\ 10 & 12 \end{pmatrix}$$

- `D=B-A` rozdiel matíc, výsledok je matica

$$D = \begin{pmatrix} 4 & 4 \\ 4 & 4 \end{pmatrix}$$

- `E=A*B` súčin matíc - násobia sa príslušné riadky a stĺpce, výsledok je matica

$$E = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}$$

- `F=A.*B` násobenie prvkov matíc A a B na rovnakých pozíciách, výsledok je matica

$$F = \begin{pmatrix} 5 & 12 \\ 21 & 32 \end{pmatrix}$$

- `G=A/B` delenie matíc sprava; matematický zápis: $A.B^{-1}$, výsledok je matica

$$G = \begin{pmatrix} 3 & -2 \\ 2 & -1 \end{pmatrix}$$

- `H=A\B` delenie matíc zľava; matematický zápis: $A^{-1}.B$, výsledok je matica

$$H = \begin{pmatrix} -3 & -4 \\ 4 & 5 \end{pmatrix}$$

- `I=A./B` delenie prvkov na rovnakých pozíciách matíc A a B, výsledok je matica

$$I = \begin{pmatrix} 0.2000 & 0.3333 \\ 0.4286 & 0.5000 \end{pmatrix}$$

- `J=A.\B` delenie prvkov na rovnakých pozíciách matíc B a A, výsledok je matica

$$J = \begin{pmatrix} 5.0000 & 3.0000 \\ 2.3333 & 2.0000 \end{pmatrix}.$$

Ďalej ukážeme použitie často používaných základných maticových funkcií. Definujme maticu $A = [1\ 2\ 3;\ 4\ 5\ 6;\ 7\ 8\ 90]$.

- `B=inv(A)` výpočet inverznej matice A^{-1} k matici A, výsledok je matica

$$B = \begin{pmatrix} -1.6543 & 0.6420 & 0.0123 \\ 1.3086 & -0.2840 & -0.0247 \\ 0.0123 & -0.0247 & 0.0123 \end{pmatrix}$$

- `C=A'` funkcia transpozície matice A, výsledok je matica

$$C = \begin{pmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 90 \end{pmatrix}$$

- `det(A)` determinant matice A, výsledok je -243
- `D=eig(A)` počíta vlastné čísla matice A, výsledok je vektor

$$D = (90.8173;\ -0.4731;\ 5.6558)$$

- `E=sum(A)` vypočíta súčty jednotlivých stĺpcov matice A, výsledok je vektor $E = (12\ 15\ 19)$. Pri aplikácii na vektor vypočíta súčet všetkých prvkov vektora.
- `F=max(A)` nájde maximálne prvky stĺpcov v matici A, výsledok je vektor $F = (7\ 8\ 90)$. Pri aplikácii na vektor nájde jeho maximálnu súradnicu. `[i,F]=max(A)` nájde maximálny prvok, ktorého hodnotu zapíše do F a do i sa zapíše index výskytu maximálneho prvku. Rovnaký syntaktický zápis má aj funkcia `F=min(A)`, ktorá hľadá minimálny prvok.
- `G=size(A)` určí veľkosť (rozmer) matice A, výsledok je vektor $G = (3\ 3)$

Okrem uvedených funkcií disponuje MATLAB aj ďalšími funkciami, ktoré sa dajú nájsť napríklad v pomocníkovi alebo zápisom príkazu `help matfun` do *Command Window*.

Pri zápise algoritmov budeme často používať maticu, ktorej prvkami budú samé nuly. Na jej zápis použijeme príkaz `zeros`. Príklad nulovej matice so štyrmi stĺpcami a tromi riadkami obsahujúcej samé nuly je nasledovný: `mtnul=zeros(4,3)`. Podobne sa dá generovať matica zo samých jednotiek príkazom `ones`. Pre prípad generovania jednotkovej matice sa používa príkaz `eye`.

9.4.2 Indexovanie matíc

Podobne ako v programovaní pri práci s prvkami polí, aj pri maticiach vieme prvky indexovať. Prvý prvok vektora má index 1 (nie 0, ako to býva v niektorých programovacích jazykoch) a ľavý horný prvok matice má súradnice (1,1). Používaním indexov v MATLABe môžeme napríklad presunúť časť jednej matice do inej matice, v rámci matice vymeniť riadky či stĺpce, zistiť hodnotu ľubovoľného prvku na daných súradniciach atď. Princíp použitia indexovania uvádzame na nasledujúcich príkladoch s vektorom $v = [13\ 5\ 8\ 78\ 81\ 8\ 3\ 10]$:

- `v(5)` vypíše hodnotu súradnice na piatej pozícii vektora v, t.j. číslo 81
- `u=v([2 7 4])` do nového vektora u zapíše postupne prvky s uvedenými indexami, t.j. $u = (5\ 3\ 78)$
- `u=v([1:3,5:8])` do nového vektora u zapíše postupne prvky s uvedenými indexami, t.j. $u = (13\ 5\ 8\ 81\ 8\ 3\ 10)$
- `v=(6:end)` vypíše prvky vektora od indexu 6 po posledný, t.j. $u = (8\ 3\ 10)$
- `v([1 5 8])=[0 4 6]` na miesta s indexami 1, 5 a 8 sa zapíšu hodnoty 0, 4 a 6, t.j. $v = [0\ 5\ 8\ 78\ 4\ 8\ 3\ 6]$

Samozrejmosťou je použitie indexácie aj pre matice. Definujme maticu `A=magic(4)`. Uvedený príkaz vytvorí štvorcovú maticu s rozmermi 4×4 s vlastnosťou, že súčet každého riadku, stĺpca aj diagonály je rovnaký (tzv. magický štvorec).

$$A = \begin{pmatrix} 16 & 2 & 3 & 13 \\ 5 & 11 & 10 & 8 \\ 9 & 7 & 6 & 12 \\ 4 & 14 & 15 & 1 \end{pmatrix}$$

Ďalej uvádzame niekoľko príkladov použitia indexácie na matici:

- `A(:,4:-1:1)` prehodí stĺpce matice do opačného poradia
- `A(:)` vytvorí stĺpcový vektor zložený zo všetkých prvkov matice A
- `A>10` vypíše maticu rovnakých rozmerov, pričom na miestach, ktoré spĺňajú uvedenú podmienku budú 1 a na ostatných miestach 0
- výmena prvého a tretieho riadku v matici A:
  `x=A(1,:);` zapamätanie si prvého riadku matice
  `A(1,:)=A(3,:);` zápis tretieho riadku do prvého riadku
  `A(3,:)=x;` zápis zapamätaného riaku do tretieho riadku v matici A

9.5 Základné algoritmické konštrukcie

Tak ako vo všetkých programovacích jazykoch, aj v MATLABe je dôležité poznať zápis základných algoritmických konštrukcií, bez ktorých sa nezaobídeme pri tvorbe vlastných programov. Vytvoriť vlastný program môžeme v prostredí MATLABu v jeho vstavanom editore, ktorý bol spomenutý v prvej kapitole.

9.5.1 Podmienený príkaz

Syntax zápisu podmieneného príkazu je nasledovná:

```
if podmienka
   príkazy
elseif podmienka
   príkazy
else
   príkazy
end
```

Príkazy sú realizované ak zadaná podmienka je pravdivá. V podmienke je povolené použiť relačné operátory <, <, ==, <=, >= ako aj logické operátory &&, ||, ~. Viacero podmienok je možné spojiť operátormi OR alebo AND. Samozrejmosťou je vytvoriť konštrukciu podmieneného príkazu podľa vlastnej potreby, t.j. podmienku neúplnú, úplnú s použitím príkazu `else` i viacnásobné testovanie pomocou príkazu `elseif`.
Príklad použitia príkazu `if` pri zisťovaní znamienka čísla:

```
if a>0
    disp('kladné'); %výpis do Command Window
elseif a<0
    disp('záporné');
else disp('nula');
end
```

Namiesto viacnásobného použitia príkazu `if-elseif...` môžeme použiť príkaz `switch`.

```
point = 85;
switch floor(point/10)
  case 9, grade = 'A'
  case 8, grade = 'B'
  case 7, grade = 'C'
  case 6, grade = 'D'
```

```
    otherwise grade = 'F'
  end
```

Poznámka: príkaz `floor(x)` zaokrúhli číslo na najbližšie celé menšie od x.

9.5.2 Príkazy cyklu

Na opakovanie jedného alebo skupiny príkazov používame v MATLABe dva druhy cyklov. Cyklus s pevným (známym) počtom opakovaní má syntax:

```
for index=dolnahranica:krok:hornahranica
   príkazy
     :
end
```

Príkaz `for` bude opakovať zadané príkazy v tele cyklu – medzi `for` a `end`. Hodnotou premennej `index` budú postupne čísla od `dolnahranica` po `hornahranica` so zadaným krokom. Ak `krok` nezadáme, bude hodnota kroku rovná 1. Hodnotu `kroku` môžeme zadať aj zápornú. Namiesto hornej a dolnej hranice môžeme zadať aj množinu hodnôt (viď druhý príklad). Cyklus obyčajne končí, ak `index` dosiahne hodnotu `hornahranica`, alebo môže byť predčasne ukončený použitím príkazu `break` vo vnútri cyklu `for`.
Príklad použitia cyklu `for` s predčasným ukončením pomocou `break` a krokom 1:

```
point = [76 85 -91 65 87];
for i = 1:length(point)
   disp(point(i));
   if point(i)<0
      break;
   end
end
```

Poznámka: príkaz `length()` zistí počet prvkov pozdĺž najväčšieho rozmeru zadaného poľa.
Príklad použitia cyklu `for` s definovanou množinou hodnôt:

```
for s = [1,5,8,17]
   disp(s)
end
```

Okrem cyklu so známym počtom opakovaní je v MATLABe možnosť použiť cyklus s neznámym počtom opakovaní. Syntax cyklu `while`:

```
while podmienka
   príkazy
      :
end
```

Príkazy sa budú opakovať dovtedy, kým bude platná podmienka cyklu. Pri zápise podmienky dodržiavame rovnaké pravidlá ako pri zápise podmienky v príkaze `if`. Podobne ako v predchádzajúcom type cyklu, aj tento vieme ukončiť predčasne použitím príkazu `break` zapísanom v tele cyklu. V prípade vzniku nekonečnej slučky (zacyklenia programu) vieme cyklus ukončiť stlačením kombinácie kláves CTRL+C.
Príklad použitia cyklu `while` pri výpočte faktoriálu:

```
n = 1;
nFactorial = 1;
while nFactorial < 100
    n = n + 1;
    nFactorial = nFactorial * n;
end
disp(nFactorial);
```

9.5.3 Funkcie v MATLABe

Pre sprehľadnenie zdrojového kódu vytváraného programu a možnosti viacnásobného volania časti kódu s meniacimi sa vstupnými parametrami používame funkcie. Syntax zápisu funkcie:

```
function [out1, out2, ...] = meno_funkcie(in1, in2, ...)
   príkazy
      :
```

Funkcie sú deklarované svojim menom, výstupnými a vstupnými parametrami. Zapisujeme ich do m-súborov, ktorých názov môže obsahovať iba povolené znaky (nesmie obsahovať znaky s diakritikou a medzery). V rámci m-súboru musí byť ako prvý uvedený príkaz `function`. Meno funkcie musí byť zhodné s menom m-súboru, v ktorom je uložená. Premenné použité v rámci funkcie sú lokálne premenné.
Príklad použitia funkcie na výpočet n členov Fibonacciho postupnosti:

```
function f=fibonacci(n)
f = zeros(n,1);
   f(1) = 1;
   f(2) = 2;
   for k = 3:n
```

```
      f(k) = f(k-1) + f(k-2);
end
```

Uvedená funkcia má jeden vstupný parameter n a jeden výstupný parameter f. V prípade, že by mala mať funkcia viac výstupných parametrov, musia byť zapísané v hranatých zátvorkách.
V programe (v inom m-súbore) potom funkciu voláme nasledovne:

```
    :
fibonacci(15)
    :
```

Výstupom takto zavolanej funkcie bude výpis postupnosti po 15. Fibonacciho číslo.

V MATLABe máme možnosť zapísať viaceré funkcie v rámci jedného m-súboru. Tieto funkcie sú však viditeľné iba pre funkcie v danom m-súbore, nie je možné ich spúšťať z iných m-súborov. Definícia takýchto funkcií sa vykoná na konci príslušného kódu v m-úbore a začína príkazom `function`. Príklad zápisu viacerých funkcií v jednom m-súbore je nasledovný:

```
function f=fibonacci(n)
   :
function [a1,a2]=bisekcia(a,b,x0)
   :
function x=gauss(A)
   :
```

9.6 Základy použitia grafiky

MATLAB predstavuje okrem výpočtov aj silný nástroj na tvorbu 2D aj 3D grafov. V nasledujúcich podkapitolách si ukážeme, ako vytvárať grafy funkcií dvoch a troch premenných.

9.6.1 Tvorba 2D grafov

Základným príkazom na vykresľovanie bodov alebo čiar je príkaz `plot`. Aby MATLAB vedel vykresliť graf funkcie, musí poznať definičný obor a funkciu.

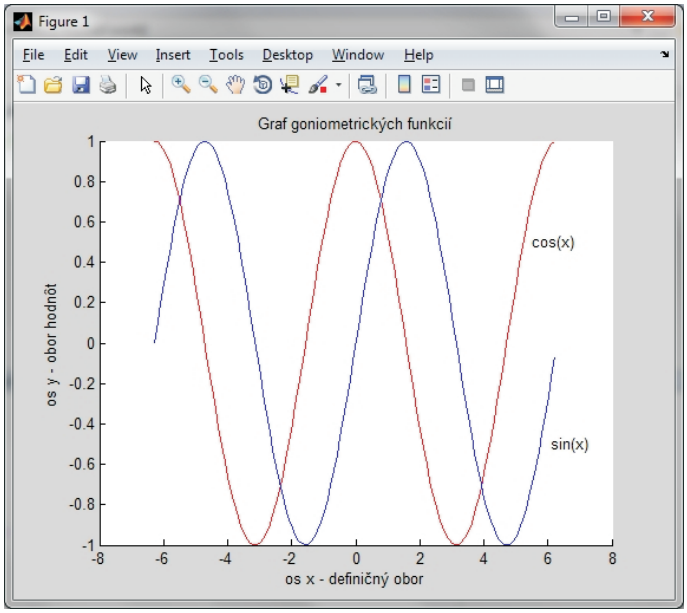
```
x=(-2*pi:0.1:2*pi); (definičný obor)
y=cos(x);           (funkcia)
```

Ďalší príkaz bude `plot(x,y);`. Po spustení programu MATLAB otvorí nové okno s obrázkom, do ktorého nakreslí modrou farbou graf funkcie kosínus na intervale

$\langle -2\pi; 2\pi \rangle$.
Ak doplníme príkazy:

```
z=sin(x);
plot(x,z);
```

tak po spustení programu bude na obrázku iba graf funkcie sínus. Je to kvôli tomu, že pri opakovanom použití príkazu `plot` sa obrázok premaže. Ak chceme tomuto zabrániť, musíme ešte pred použitím príkazov `plot` zapísať príkaz `hold on`, ktorý zabráni premazávaniu resp. prekreslovaniu obrázku. Vypnúť túto funkciu môžeme príkazom `hold off`.



Obr. 9.2: 2D graf funkcií sin(x) a cos(x)

Pre lepšiu prehľadnosť môžeme nastaviť pre dané krivky rôzne farby, štýly a typy bodov pri vykresľovaní. To docielime tak, že doplníme príkaz `plot` o textový parameter:

```
plot(x,y,'r');
```

Príkaz vykreslí graf funkcie červenou farbou. Pre zmenu štýlu iba doplníme do apostrofov napríklad „- -“ a čiara bude čiarkovaná:

```
plot(x,y,'--');
```

Samozrejmosťou je aj kombinácia farieb, štýlov a typov bodov:

```
plot(x,y,'yx');
```

Týmto príkazom sa vykreslí graf funkcie vykrížikovaný žltou farbou.
V nasledujúcej tabuľke je prehľad použitia znakov pre nastavenie štýlu, farby a bodu príkazu `plot`.

Farba	**Body**	**Krivka**
b (modrá)	. (bod)	- (plná)
c (tyrkysová)	o (krúžok)	: (bodkovaná)
g (zelená)	x (krížik)	-. (bodkočiarkovaná)
k (čierna)	+ (krížik)	- - (prerušovaná)
m (fialová)	* (hviezdička)	
r (červená)	d (hviezda)	
y (žltá)	s (štvorec)	
	^ (trojuholník nahor)	
	v (trojuholník nadol)	
	< (trojuholník vľavo)	
	> (trojuholník vpravo)	
	p (päťuholník)	
	h (šesťuholník)	

Na zobrazenie mriežky použijeme príkaz `grid on;`, na jej vypnutie `grid off;`. Ďalej uvedieme niekoľko ďalších príkazov, ktorými nastavujeme popis grafu:

- `xlabel('os x')` nastaví popis osi x
- `ylabel('os y')` nastaví popis osi y
- `title('Graf goniometrických funkcií')` nastaví popis názvu (titulok) grafu
- `text(3,4.5,'text v grafe')` umiestni text v apostrofoch na súradnice v grafe
- `gtext('text po kliknutí')` umiestni text v apostrofoch na miesto po kliknutí myšou na graf

Graf vytvorený použitím uvedených príkazov je na obrázku 9.2.

9.6.2 Tvorba 3D grafov

Vytváranie trojrozmerných grafov je podobné tvorbe grafov dvojrozmerných. Väčšina príkazov je analogická s príkazmi z predchádzajúcej kapitoly. Začneme klasickým príkladom grafu špirály danej parametricky:

```
t=(0:pi/45:15*pi);
plot3(sin(t),cos(t),t)
```

Na kreslenie špirály bol v predchádzajúcom príklade použitý príkaz `plot3(X,Y,Z)`, ktorého parametre X, Y a Z sú vektory. Pre tento príkaz je možné použiť rovnaké rozširujúce parametre pre farbu, štýl a body ako pre príkaz `plot` opísané v predchádzajúcej podkapitole. Príkazy pre popis grafu (názvy osí, názov grafu a pod.) sú podobné s príkazmi pre 2D grafy.
Okrem spojitého čiarového grafu sa dá v MATLABe vykresliť aj plošný 3D graf. Najskôr musíme vytvoriť mriežku (maticu), ktorá bude základom na vykreslenie 3D grafu. Vymedzíme si na osiach x a y intervaly, ktoré budú vymedzovať časť vykreslenia grafu funkcie dvoch premenných.

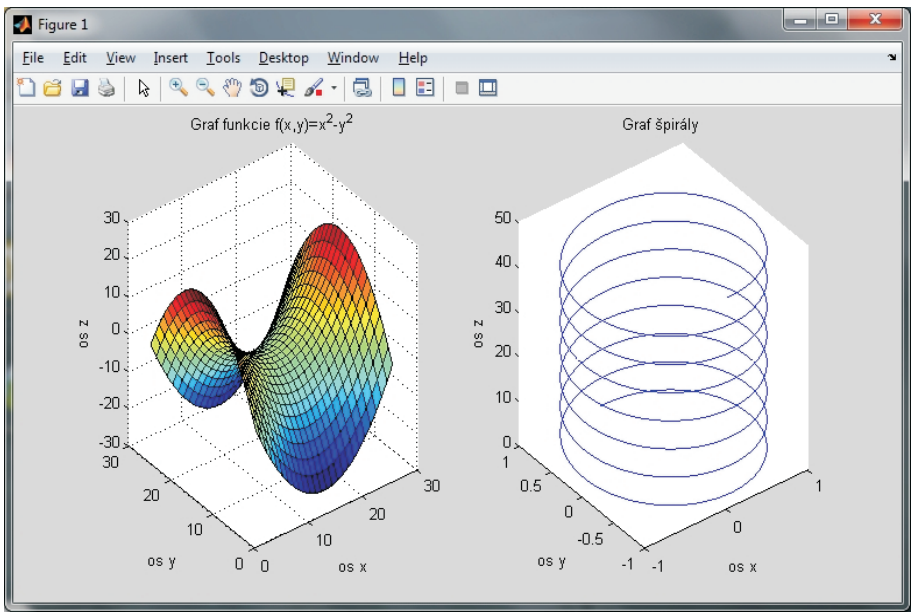
```
X=-5:0.1:5;
Y=-5:0.1:5;
[X,Y]=meshgrid(X,Y); %vytvorenie mriežky resp. matice
Z=X.^ 2-Y.^ 2;   %výpočet Z-ových súradníc bodov mriežky
mesh(Z)              %vykreslenie 3D grafu
```

Príkazom `mesh` sa vykreslí sieťový graf, na ktorom sú farebne odlíšené funkčné hodnoty Z. Pri zmene príkazu na `meshc` sa vykreslí rovnaký graf doplnený o vrstevnicové grafy. Ak použijeme príkaz `surf(Z)`, tak výsledný graf bude mať plochy siete vyfarbené.
Zmenu pohľadu grafu môžeme meniť pomocou ikony na paneli obrázku (Rotate 3D), alebo príkazom

```
view(alpha,beta)
```

Parameter *alpha* predstavuje uhol azimutu (odklon od osy y) a parameter *beta* uhol elevácie (odklon od osy x – pohľad pozorovateľa). Implicitné nastavenie týchto hodnôt je $alpha = -37.5°$ a $beta = +30°$. Pri zmene pohľadu na obrázok pomocou myši sa hodnoty parametrov zobrazujú v ľavom dolnom rohu okna.

Ak potrebujeme vykresliť do jedného okna viacero grafov vedľa seba, môžeme tak urobiť pomocou príkazu `subplot(a,b,c)`. Tento príkaz rozdelí okno obrázku na rovnaké časti. Parameter a udáva počet obrázkov zobrazených vodorovne, parameter b znamená počet obrázkov zobrazených zvisle a c je parameter, ktorý určuje

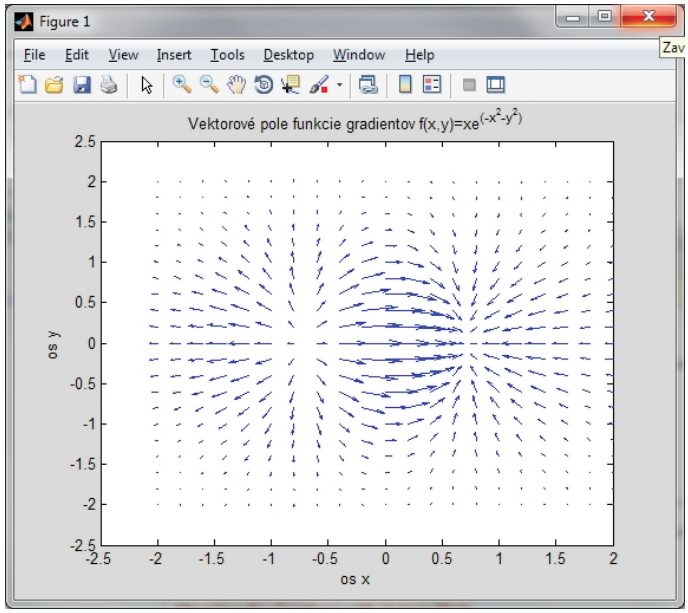


Obr. 9.3: Ukážka použitia príkazu subplot

pozíciu obrázku. Indexy obrázkov sú číslované po riadkoch z ľava do prava a prislúcha im parameter c. Príklad použitia príkazu `subplot` je zobrazený na obrázku 9.3 na strane 175.

MATLAB poskytuje zobrazenie aj iných typov grafov. Pomocou príkazu `quiver(x,y,u,v)` dokážeme vykresliť vektorové pole. Príkaz vykresľuje vektory ako šípky na súradniciach tvorených všetkými dvojicami prvkov matíc x a y. Matice x, y, u a v musia byť rovnakej veľkosti a obsahovať zodpovedajúce polohy a súradnice vektorov.
Pridaním ďalšieho parametra `quiver(....,scale)` vieme meniť dĺžku vykresľovaných vektorov. Ak $scale < 1$ veľkosť vektorov zmenšíme, ak $scale > 1$ veľkosť vektorov zväčšíme. Nasledujúca časť kódu demonštruje použitie príkazu `quiver` a grafický výstup je zobrazený na obrázku 9.4.

```
[X,Y]=meshgrid(-2:0.1:2);
Z=X.*exp(-X.^2-Y.^2);  %výpočet Z-ových súradníc
[DX,DY]=gradient(Z,0.2,0.2);  %výpočet gradientov
quiver(X,Y,DX,DY,1.5); %vykreslenie vektorového poľa
title('Vektorové pole gradientov pre xe^{(-x^2-y^2)}');
xlabel('os x');
ylabel('os y');
```



Obr. 9.4: Ukážka vykreslenia vektorového poľa gradientov

9.7 Zapisovanie a načítavanie dát

MATLAB poskytuje zapisovanie dát do súborov a tiež ich načítavanie. Dáta je možné zapisovať nielen do textových súborov, ale tiež do súborov tabuľkového procesora MS Excel. Najskôr sa budeme venovať zapisovaniu dát do uvedených typov súborov, akým spôsobom zapisované dáta formátovať a nakoniec budeme dáta zo súborov načítavať a ďalej s nimi pracovať.

9.7.1 Formátovanie a ukladanie dát

Údaje, s ktorými pracujeme nemusia byť vždy zobrazované tak, ako by sme očakávali. MATLAB umožňuje výstupné dáta formátovať do podoby zvolenej používateľom. V nasledujúcej časti ukážeme, akým spôsobom formátovať dáta a zapisovať ich do textovej premennej a do súboru.

Pomocou príkazu `sprintf` môžeme čísla alebo reťazce zapisovať do premennej typu string. Použitie príkazu vysvetlíme na príklade:

```
s=sprintf('%010.5f',pi);
```

Vysvetlenie k príkazu:

- % – začiatočný povinný znak,

- 0 – prepínač, jednotlivé prepínače sú uvedené ďalej v tabuľke,
- 10 – šírka poľa odsadenia, t.j. počet prázdnych znakov zľava pred dátami,
- 5 – presnosť (počet desatinných miest); pred presnosť treba vždy písať bodku,
- f – konverzný znak, ktorý určuje formát uvedený v tabuľke.

Sformátovaný výstup potom bude nasledovný `'0003.14159'`.
V nasledujúcej tabuľke sú uvedené vysvetlenia ku špeciálnym znakom.

Znak zmeny	**Význam**
\" alebo "	znak úvodzoviek
%%	znak percent
\\	spätné lomítko
$\backslash b$	návrat na predchádzajúci riadok
$\backslash f$	posun formulára
$\backslash n$	nový riadok
$\backslash r$	odriadkovanie
$\backslash t$	horizontálny tabulátor
$\backslash v$	vertikálny tabulátor

V ďalšej tabuľke uvádzame zoznam konverzných znakov.

Konverzný znak	**Význam**
$\%d$ alebo $\%i$	desiatkové zobrazenie so znamienkom
$\%u$	desiatkové zobrazenie bez znamienka
$\%o$	zobrazenie v osmičkovej sústave
$\%x$	hexadecimálne zobrazenie s malými písmenami
$\%X$	hexadecimálne zobrazenie s veľkými písmenami
$\%f$	zobrazenie s pevnou rádovou čiarkou
$\%e$	exponenciálne zobrazenie s malým e
$\%E$	exponenciálne zobrazenie s veľkým E
$\%g$	podobne ako %e a %f, ale vypustia sa nevýznamné nuly
$\%G$	ako %g ale s veľkým E
$\%c$	jeden znak
$\%s$	reťazec znakov

V nasledujúcej tabuľke sú uvedené prepínače.

Značka	**Význam**
'−'	zarovnanie vľavo
'+'	výpis znamienka || ' '	medzera - pred číslo sa vloží medzera
'0'	doplnenie nulami z ľavej strany

9.7.2 Zápis dát do textového súboru

Formátovať dáta a zapísať ich do súboru vieme príkazom `fprintf`. Jeho použitie je demonštrované v nasledujúcom kóde. Prvými tromi riadkami vytvoríme dáta, ktoré sa budú do súboru zapisovať. Pomocou príkazu `fopen` sa vytvorí a otvorí nový textový súbor. Parameter *'data.txt'* hovorí o umiestnení a názve súboru, druhý parameter *'w'* o tom, že sa má vytvoriť nový súbor a v prípade už existujúceho súboru sa starý prepíše novými dátami. Podrobnejšie informácie o druhom parametre nájde čitateľ v pomocníkovi MATLABu.

```
x=0:4*pi/10:4*pi;
y=sin(x.^2);
data=[x,y];
subor=fopen('data.txt','w');
fprintf(subor,'%0.3f %5.8f\n',data);
fclose(subor);
```

Parametre príkazu `fprintf` sú zrejmé z jeho zápisu v uvedenom kóde. Ako prvý parameter sa zapisuje súbor, do ktorého sa zapíšu dáta a ako druhý parameter je zapísaný formát zápisu dát, ktorého forma je zhodná s príkazom `sprintf`. Posledným, tretím parametrom je premenná obsahujúca dáta, ktoré sa majú sformátovať a zapísať do súboru. Nakoniec treba otvorený súbor uzavrieť príkazom `fclose`.

9.7.3 Zápis dát do XLS súboru

Na export dát do xls súboru používame príkaz `xlswrite`. Syntax zápisu je nasledovná:

```
xlswrite(filename, M, sheet, range);
```

Povinné parametre sú `filename` a `M`. `M` musí byť matica, ktorej rozmer neprevyšuje maximálny počet riadkov a stĺpcov hárku v xls súbore. Obsah premennej môže byť číselný, textový alebo typu *cell*. Ak nezadáme paramater `sheet`, ktorý predstavuje názov hárku, do ktorého sa bude zapisovať a ani parameter `range`, ktorý predstavuje oblasť zápisu v rámci hárku, údaje sa zapíšu do prvého hárku v xls súbore od pozície A1.
Názov hárku zadávame vo forme reťazca. Ak hárok so zadaným menom v xls súbore neexistuje, pridá sa hárok so zadaným názvom na koniec xls súboru a do *Command window* sa vypíše hlásenie o pridaní hárku. Pre lepšie pochopenie uvádzame príklad zápisu hodnôt do xls súboru s názvom *myData.xls*.

```
values={1, 2, 3, 4; 5, 6, 'a', 'b'; 9, 10, 11, 12};
headers={'First', 'Second', 'Third', 'Fourth'};
xlswrite('myData.xls', [headers; values]);
```

Výstup do xls súboru je zobrazený na nasledujúcom obrázku 9.5.

	A	B	C	D
1	First	Second	Third	Fourth
2	1	2	3	4
3	5	6	a	b
4	9	10	11	12

Obr. 9.5: Dáta zapísané v xls súbore príkazom `xlswrite`

9.7.4 Načítanie dát cez Command Window

Načítanie vstupných hodnôt do programu nemusíme vždy realizovať priamym zápisom do premennej v zdrojovom kóde. MATLAB poskytuje možnosť načítať hodnotu do premennej z príkazového riadku príkazom `input`. Na jednoduchom príklade ukážeme jeho použitie:

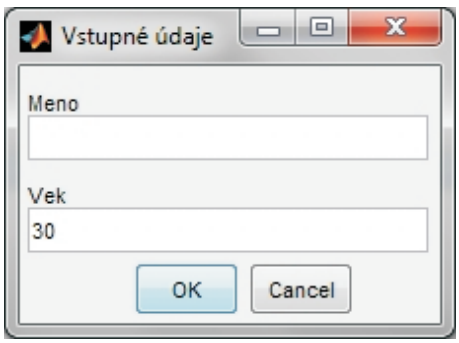
```
evalResponse=input('Zadaj hodnotu: ');
strResponse=input('Zadaj retazec: ','s');
```

Po spustení tejto sekvencie príkazov bude v *Command Window* vypísaný reťazec v apostrofoch a MATLAB bude čakať na zadanie číselnej hodnoty, ktorú po potvrdení zapíše do premennej `evalResponse`. Rovnaký postup prebehne aj pri druhom príkaze, kedy však na vstupe MATLAB očakáva nie číselnú, ale reťazcovú hodnotu, čo je prednastavené parametrom `'s'` v príkaze `input`.

9.7.5 Načítanie dát prostredníctvom dialógových okien

Prehľadnejší spôsob načítavania vstupných údajov poskytuje MATLAB príkazom `inputdlg`, ktorý vytvorí počas behu programu modálne dialógové okno s príslušným počtom editovacích políčok pre zadanie vstupných údajov. Parametre príkazu a jeho použitie vysvetľujeme na príklade a dialógové okno je zobrazené na obrázku 9.6.

```
prompt = {'Meno', 'Vek'};
dlg_title = 'Vstupné údaje';
num_lines = 1;
def_ans = {'','30'};
answer = inputdlg(prompt,dlg_title,num_lines,def_ans)
```



Obr. 9.6: Ukážka dialógové okna vytvoreného príkazom `inputdlg`

Ako vidieť v zdrojovom kóde, príkaz `inputdlg` je zapísaný ako funkcia s parametrami. Do prvého parametra `prompt` zapisujeme reťazce, ktoré budú zobrazené ako popis editovacích políčok v dialógovom okne. Počet zadaných reťazcov oddelených čiarkami určuje počet vygenerovaných editovacích políčok. Titulok dialógového okna zapíšeme ako reťazec do parametra `dlg_title`. Počet riadkov v editovacích políčkach je nastavený tretím parametrom `num_lines`. Ak chce používateľ niektorým editovacím políčkam prednastaviť hodnoty, zapíše ich do štvrtého parametra `def_ans`.
V našom príklade vystupujú štyri parametre, ale povinný je iba prvý z nich - `prompt`. Ostatné parametre sú voliteľné, nie je potrebné ich zapisovať pre každé dialógové okno. Jednotlivé hodnoty parametrov je možné zapísať aj priamo ako parametre príkazu `inputdlg` bez použitia premenných, avšak vtedy sa stáva zápis menej prehľadný. Podrobnejšie informácie o použití príkazu `inputdlg` nájde čitateľ v pomocníkovi MATLABu.

9.7.6 Načítanie dát z textového súboru

Príkaz `importdata` používame na načítavanie dát z textových súborov. Syntax zápisu príkazu je nasledovná:

```
S=importdata(filename, delimeter, nheaderlines);
```

Parameter `filename` určuje cestu a meno súboru, z ktorého sa budú dáta čítať. Druhý parameter `delimeter` reprezentuje separátor (oddeľovač), ktorým sú oddelené stĺpce s dátami v ASCII súbore. Týmto znakom môže byť čiarka „,“ alebo bodkočiarka „;“. Ak sú dáta oddelené tabulátorom, ako separátor treba zapísať „\t“. `nheaderlines` udáva počet riadkov, ktoré zaberá hlavička načítavaných dát v ASCII súbore. Príklad načítania dát zo súboru *myfile.txt*, kde oddelovačom je medzera a prvý riadok v súbore obsahuje hlavičku je nasledovný:

```
M = importdata('myfile.txt', ' ',1);
```

Dáta sú načítané do premennej `M`, ktorá obsahuje ako hlavičku, tak aj samotné dáta. Pracovať samostatne s dátami a s hlavičkou vieme pomocou premenných `M.data` a `M.colheaders`. Nech súbor *myfile.txt* obsahuje takéto dáta:

```
Day1  Day2  Day3  Day4  Day5  Day6  Day7
95.01 76.21 61.54 40.57  5.79 20.28  1.53
23.11 45.65 79.19 93.55 35.29 19.87 74.68
60.68  1.85 92.18 91.69 81.32 60.38 44.51
48.60 82.14 73.82 41.03  0.99 27.22 93.18
89.13 44.47 17.63 89.36 13.89 19.88 46.60
```

Po použití vyššie uvedeného príkazu sa jeho obsah načíta do premennej `M`. Pre výpis napríklad tretieho stĺpca aj s príslušnou hlavičkou použijeme príkazy:

```
disp(M.colheaders(:,3));
disp(M.data(:,3));
```

9.7.7 Načítanie dát z XLS súboru

Skôr než sa budeme venovať čítaniu dát z xls súboru, poviemem si o príkaze `xlsfinfo`, ktorý zistí, či daný súbor obsahuje hárky MS Excelu. Syntax zápisu príkazu je nasledovná:

```
[typ, desc] = xlsfinfo('myData.xls');
```

Ak je súbor rozoznaný ako súbor MS Excelu, do premennej `typ` je zapísaný reťazec *Microsoft Excel Spreadsheet* a do premennej `desc` sa zapíšu názvy hárkov xls súboru. Ak nebude súbor rozoznaný ako xls formát, do premennej `typ` sa zapíše prázdny reťazec a do premennej `desc` reťazec *Unreadable xls file*. Týmto spôsobom vieme predísť chybám vznikajúcim pri čítaní nesprávneho súboru.

Pre čítanie údajov z xls súboru používame príkaz `xlsread`. Rovnako aj tento príkaz je možné zapísať s viacerými parametrami. Syntax jeho zápisu je nasledovná:

```
[num, txt]=xlsread(file, sheet, range);
```

Prvý parameter `file`, ktorý predstavuje názov resp. cestu k xls súboru je povinný, ďalšie dva nie sú povinné. Keď zadáme iba prvý parameter (meno súboru), tak do premennej `num` sa zapíšu všetky čísla z prvého hárku xls súboru a ostatné znaky sú ignorované a nahradené hodnotou `NaN`. Ak chceme čítať údaje z konkrétneho hárku, doplníme parameter `sheet` zapísaný buď ako reťazec s názvom hárku alebo

ako číslo predstavujúce poradové číslo hárku v xls súbore.
Zadaním tretieho parametra `range` určíme oblasť, z ktorej sa budú údaje čítať. Oblasť zadávame v podobe reťazca a jej zápis je rovnaký ako v MS Excel.
Štandardne sa z xls súboru čítajú číselné údaje. Ak však chceme načítať aj reťazce, musíme pridať druhý výstupný parameter `txt`, do ktorého sa tieto údaje zapíšu, ale číselné údaje budú skonvertované na prázdny reťazec.

9.8 Index použitých príkazov MATLABu

Zápis	Strana	Popis
`break`	169	preruší cyklus
`clear` all	163	vymaže obsah všetkých použitých premenných
`clear` premenná	163	vymaže obsah premennej
`det()`	166	vypočíta determinant matice
`disp()`	168	vypisuje text alebo pole
`diff()`	183	počíta rozdiely medzi susednými prvkami v poli
`eig()`	166	vypočíta vlastné čísla matice
`eye()`	166	generuje jednotkovú maticu
`fclose()`	178	zatvorí otvorený súbor
`floor()`	169	zaokrúhli číslo na najblžšie menšie celé číslo
`fminsearch()`	183	viacparametrická nelineárna optimalizácia bez ohraničení Nelderovou-Meadovou metódou
`fmincon()`	183	viacparametrická nelineárna optimalizácia s ohraničeniami
`fopen()`	178	vytvorí a otvorí súbor pre zapisovanie
`for`	169	cyklus so známym počtom opakovaní
`format long`	161	nastaví počet desatinných miest na 14
`format short`	161	nastaví počet desatinných miest na 4
`fprintf()`	178	formátuje údaje a zapisuje ich do súboru
`function`	170	príkaz pre tvorbu podprogramov - funkcií
`gradient()`	175	výpočet gradientov funkcie
`grid on/off`	173	povolí/zakáže zobrazovanie mriežky v grafe
`gtext()`	173	umiestni text v grafe na súradnice po kliknutí na graf
`help`	166	zobrazí pomocníka MATLAB
`hist()`	183	kreslí histogram zo zadaných dát
`hold on/off`	172	povolí/zakáže kresliť viac grafov do jedného grafického okna
`if...then`	168	príkaz podmienky
`importdata()`	180	číta údaje z textového súboru
`input()`	179	číta vstupné údaje z Command Window

Zápis	Strana	Popis
`inputdlg()`	179	vytvorí dialógové okno pre načítanie údajov
`interp1`	184	interpoluje 1-rozmerné dáta; možnosť výberu metódy: nearest, linear, spline, pchip, cubic
`interp2`	184	interpoluje 2-rozmerné dáta; možnosť výberu metódy: nearest, linear, spline, cubic
`inv()`	166	vypočíta inverznú maticu
`isnumeric()`	184	zistí, či je vstup číselné pole
`length()`	169	zistí počet prvkov pozdĺž najväčšieho rozmeru zadaného poľa
`linprog()`	184	rieši problém lineárneho programovania
`lscov()`	184	metóda najmenších štvorcov s váhou so známou chybou kovariancie
`lsqcurvefit()`	184	metóda najmenších štvorcov s váhou - spojitý prípad
`lsqlin()`	184	rieši metódu najmenších štvorcov - lineárny prípad
`lsqnonlin()`	184	rieši metódu najmenších štvorcov - nelineárny prípad
`magic()`	167	generuje maticu, v ktorej platí, že súčet prvkov v riadkoch, stĺpcoch aj na diagonále je rovnaký
`max()`	166	nájde maximálne prvky stĺpcov matice resp. maximálnu súradnicu vektora
`mesh()`	174	kreslí 3D graf v podobe štvorcovej siete
`meshc()`	174	kreslí 3D graf v podobe štvorcovej siete s vrstevnicami
`meshgrid()`	174	generuje X a Y pole pre 3D vykresľovanie
`min()`	166	nájde minimálne prvky stĺpcov matice resp. minimálnu súradnicu vektora
`mod()`	184	počíta zvyšok po celočíselnom delení
`norm()`	184	počíta normu vektora/matice
`ode23()/ode45()`	184	riešiteľ obyčajných diferenciálnych rovníc
`ones()`	166	generuje maticu zo samých jednotiek
`plot()`	171	kreslí 2D graf zadaný pomocou hodnôt vo vektore

Zápis	Strana	Popis
`plot3()`	174	kreslí 3D graf zadaný pomocou hodnôt vo vektore
`polyder()`	185	počíta deriváciu polynómu
`polyint()`	185	počíta integrál polynómu
`polyval()`	185	počíta hodnotu polynómu
`quad()`	185	numericky počíta integrál Simpsonovou metódou
`quiver()`	175	kreslí vektorové pole
`rand()`	185	generátor náhodných čísel
`randn()`	185	Gaussov generátor náhodných čísel
`round()`	185	zaokrúhľuje reálne číslo na najbližšie celé číslo
`size()`	166	vypočíta veľkosť (rozmer) matice
`solve()`	185	rieši rovnice a systémy rovníc
`spline()`	185	interpoluje dáta kubickým splajnom
`sprintf()`	176	formátuje údaje a zapisuje do reťazca
`subplot()`	174	nastaví kreslenie viacerých grafov do jedného okna
`sum()`	166	vypočíta súčty stĺpcov matice, resp. súčet prvkov vektora
`surf()`	174	kreslí 3D graf v podobe tieňovaných plôch
`switch`	168	príkaz pre viacnásobné vetvenie
`text()`	173	umiestni text v grafe na dané súradnice
`title()`	173	nastavuje popis názvu grafu
`view()`	174	zmena pohľadu na graf
`while`	169	cyklus s neznámym počtom opakovaní
`xlabel()`	173	nastavuje popis osi x v grafe
`xlsfinfo()`	181	zistí, či je daný súbor typu MS Excel
`xlsread()`	181	číta údaje z xls súboru
`xlswrite()`	178	zapíše údaje do xls súboru
`ylabel()`	173	nastavuje popis osi y v grafe
`zeros()`	166	generuje nulovú maticu